

ext_sum_correction — Orphan MAC Correction Module

Module Documentation — Stage 1 Adder-Tree Support / MultiStage-DeepSNN

Module name	ext_sum_correction
Purpose	Rescues the 24 “orphan” MAC products left over by the 12 Stage-1 adder trees and turns them into 8 additional valid conv25 partial sums.
Written by	Unknown (original author not recorded)
Revised by	Mohamed Hussein
Date	July 6, 2026

1. Purpose

This module exists purely to clean up after the 12 Stage-1 adder trees (adder_tree_10_4_1_1). Its output is not a separate feature — it is 8 additional, correctly-normalized partial sums that get used exactly like the trees' own Layer-1 taps.

2. The Core Problem It Solves (the “orphan” MACs)

Looking at the bigger picture from the project's module spec: there are 12 adder trees, and each tree receives 32 MAC outputs from the convolution array. Inside each tree, MACs 0 to 29 (30 MACs) are grouped into 10 taps of 3 MACs each, giving 10 normal outputs per tree.

MACs 30 and 31 in every tree are left over — they don't fit into a group of 3 within their own tree. These are the “orphans.”

Stage 1 needs exactly 128 inputs to feed 32 Shaaban units $\times$ 4 inputs each. The 12 trees give $12 \times 10 = 120$ normal taps, leaving $128 - 120 = 8$ additional sums needed. Since $12 \text{ trees} \times 2 \text{ orphans each} = 24$ orphan MACs, and $8 \text{ correction sums} \times 3 \text{ MACs each} = 24$, this module exists specifically to group those 24 orphans into 8 valid 3-MAC sums.

Quantity	Value	Note
Adder trees	12	adder_tree_10_4_1_1 instances
MAC inputs per tree	32	from the convolution array
Normal taps per tree	10	MACs 0–29, grouped by 3
Orphans per tree	2	MACs 30 and 31
Total orphans	24	$12 \text{ trees} \times 2$
Stage 1 inputs needed	128	$32 \text{ Shaaban units} \times 4 \text{ inputs}$
Normal taps supplied	120	$12 \text{ trees} \times 10$
Correction sums needed	8	$128 - 120$

3. The Mapping Logic (why the grouping looks “sequential”)

The module takes the entire 2D array `mac_in[12][32]` and forms a flat, sequential pool of the 24 orphans, indexed 0 to 23:

```
pool_index i (0 to 23) -> tree = i/2, port = 30 + (i%2)

pool[0] = Tree 0, Port 30
pool[1] = Tree 0, Port 31
pool[2] = Tree 1, Port 30
...
pool[23] = Tree 11, Port 31
```

It then simply groups this pool sequentially into 8 groups of 3. Because each tree contributes exactly 2 orphans to the flat pool, most groups of 3 end up straddling a tree boundary — borrowing the next tree's first orphan to complete a group. This is why the grouping looks like it “jumps” between trees rather than staying within one:

Correction adder	Orphan MACs used
<code>corr_out[0]</code>	<code>pool[0]+pool[1]+pool[2] = (T0,P30) + (T0,P31) + (T1,P30)</code>
<code>corr_out[1]</code>	<code>pool[3]+pool[4]+pool[5] = (T1,P31) + (T2,P30) + (T2,P31)</code>
<code>corr_out[2]</code>	<code>pool[6]+pool[7]+pool[8] = (T3,P30) + (T3,P31) + (T4,P30)</code>
<code>corr_out[3]</code>	<code>pool[9]+pool[10]+pool[11] = (T4,P31) + (T5,P30) + (T5,P31)</code>
<code>corr_out[4]</code>	<code>pool[12]+pool[13]+pool[14] = (T6,P30) + (T6,P31) + (T7,P30)</code>
<code>corr_out[5]</code>	<code>pool[15]+pool[16]+pool[17] = (T7,P31) + (T8,P30) + (T8,P31)</code>
<code>corr_out[6]</code>	<code>pool[18]+pool[19]+pool[20] = (T9,P30) + (T9,P31) + (T10,P30)</code>
<code>corr_out[7]</code>	<code>pool[21]+pool[22]+pool[23] = (T10,P31) + (T11,P30) + (T11,P31)</code>

In the RTL this mapping is generated automatically per correction adder `c` (0 to 7) rather than hand-listed: `global index gi = 3c + {0,1,2}, tree = gi/2, port = 30 + (gi%2).`

4. Arithmetic: Sum and Normalize

Each correction adder sign-extends its three 18-bit signed orphans to 20 bits (2 guard bits, enough headroom for three signed 18-bit terms), sums them, then right-shifts the 20-bit result by 1 bit and keeps the lower 18 bits as the final output. The right-shift matches the same $\div 2$ normalization already applied to the adder trees' own Layer-1 taps, so `corr_out[]` is directly comparable to — and usable alongside — the trees' normal outputs.

```
raw_sum[c] = sext(pool_a, 20) + sext(pool_b, 20) + sext(pool_c, 20)
corr_out[c] = raw_sum[c][18:1] // normalize (/2) + truncate to 18 bits
```

5. Post-Synthesis Schematic

For reference, the synthesized netlist for this module (Vivado) is shown below — the 8 correction adders and their orphan-MAC fan-in are visible as the repeated structures across the schematic.

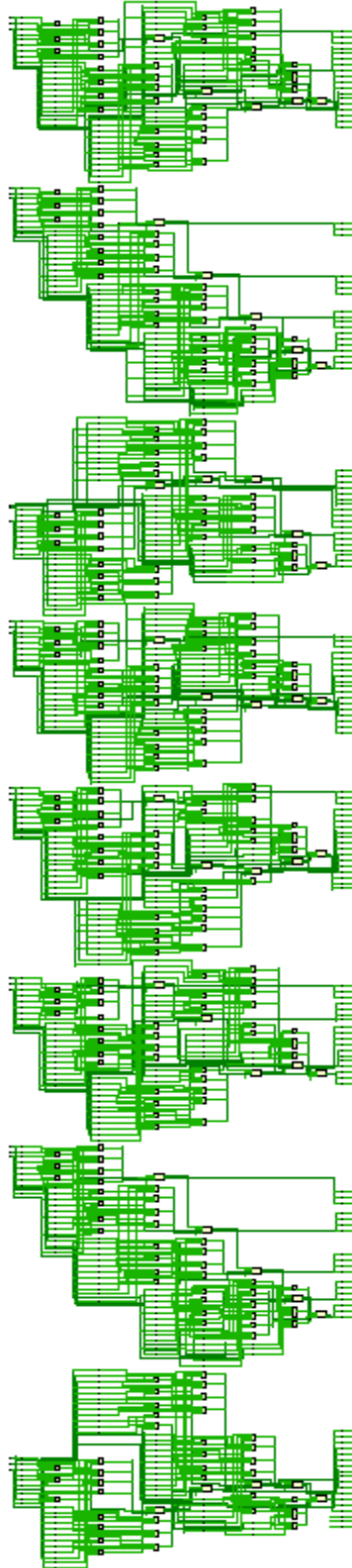


Figure 1. Post-synthesis schematic of `ext_sum_correction` (Vivado).

6. Port Description

Signal	Dir.	Width	Description
mac_in	input	[12][32][18] signed	Full MAC array from the 12 adder trees; this module reads only ports 30 and 31 of each tree.
corr_out	output	[8][18] signed	8 corrected, normalized partial sums — used alongside the trees' 120 normal taps to make up Stage 1's 128 required inputs.

Parameters

Parameter	Default	Description
N_TREES	12	Number of Stage-1 adder trees; must match the top-level instantiation.
N_CORRECTION	8	Number of correction sums = $N_SHAABAN \times INPUTS_PER_SHB - N_TREES \times TAPS_PER_TREE$.
DATA_WIDTH	18	Signed fixed-point data width.

7. Implementation Notes

- Purely combinational — no clock, no state, no always blocks. No always_ff/always_comb conversion was applicable to this module.
- Ports and internals already used the logic type in the original source, so no wire/reg → logic conversion was needed.
- Added the module header block and staged ('===') comments before each critical stage: the orphan index → (tree, port) mapping, the three-input sign-extended adder, and the normalization/truncation step.

8. Revision Log

Date	Author	Change
July 5, 2026	Mohamed Hussein	Added module header block; added staged ('===') comments before each critical stage. No wire/reg or always-block conversion needed (module already used logic, no sequential logic present).
July 6, 2026	Mohamed Hussein	Authored this documentation, covering the orphan-MAC problem, the pool-index mapping logic, the correction arithmetic, and the post-synthesis schematic.